

Biblioteca Cordillera — Documentación técnica y Plan de acción

Manual de referencia del proyecto: qué es, cómo está hecho, cómo funcionan los pagos, qué tecnologías y servicios usa, cuánto cuesta, y un plan para crecer en buscadores.

PARTE 1 — SEO (posicionamiento en buscadores)

¿Qué es el SEO y para qué sirve?

SEO (Search Engine Optimization) es el conjunto de técnicas para que tu sitio ****aparezca y se ubique bien**** en buscadores como Google. El objetivo: que cuando alguien busque "comprar libros usados Perú" (o el título de un libro), tu tienda aparezca y reciba visitas **gratis** (tráfico orgánico), sin pagar publicidad.

El proceso de Google tiene tres etapas: **rastrear** (leer tu sitio) → **indexar** (guardarlo) → **rankear** (ubicarlo en los resultados).

¿Cómo se usa y se configura? (lo que aplicamos)

- **Títulos y descripciones** únicos por página (lo que se ve en el resultado de Google).
- **Open Graph**: define la portada/título que aparece al compartir un link en WhatsApp/redes.
- **robots.txt**: le dice a Google qué puede rastrear (y bloquea áreas privadas).
- **sitemap.xml dinámico**: lista automática de todas las URLs (incluye cada libro) para que

Google las encuentre.

- **Datos estructurados (Schema.org)**: describen cada libro como "producto" (precio, stock)

para resultados enriquecidos.

- **Prerender por libro**: como el sitio se arma con JavaScript, un proceso en el servidor

inyecta los datos de cada libro en el HTML para que los buscadores y redes los lean bien.

- **URLs canónicas**: evitan que Google penalice contenido duplicado.

Plan de acción SEO (sostenible)

Cada semana

- Cargar **libros nuevos** con sinopsis **propia** (original, no copiada de la contratapa).
- Compartir 2–3 libros en **redes sociales** con el link (usa el botón "Compartir").
- Revisar que las fichas tengan **portada** y datos completos.

Cada mes

- Entrar a **Google Search Console** y revisar: páginas indexadas, errores, y por qué búsquedas aparecen.
- Conseguir **1–2 enlaces** (backlinks): grupos de lectura, foros, blogs amigos, etc.
- Actualizar precios/stock (mantener el sitio "fresco").

Una sola vez (pendiente)

- Registrar un **dominio propio** (lo que más mejora el posicionamiento).
- Verificar el sitio en **Search Console** y **Bing Webmaster Tools**, y enviar el sitemap.
- Crear un **Google Business Profile** si hay punto físico/zona.

Regla de oro: el SEO **madura en semanas/meses**. La constancia (contenido + difusión) es lo que mueve la aguja. Las ventas en sí no posicionan (Google no las ve); lo que cuenta es el **contenido nuevo, el tráfico y los enlaces**.

PARTE 2 — Estructura de la plataforma

El proyecto es un **monorepo** (un solo repositorio con varias partes):

```
Proyecto Biblioteca Cordillera/  
■ apps/  
■ ■ web/    → la página web (lo que ve el usuario)  
■ ■ api/    → el servidor / cerebro (lógica y datos)  
■ packages/  
■ ■ db/     → el esquema de la base de datos (compartido)  
■ docs/     → documentación (este archivo)
```

Cómo fluye una acción (ej. ver un libro):

1. El **navegador** carga la **web** (React) desde Cloudflare Pages.
2. La web le pide datos a la **API** (Hono en Cloudflare Workers).
3. La API consulta la **base de datos** (Cloudflare D1) y devuelve la info.
4. Las **imágenes** se sirven desde **Cloudflare R2**.

Es una arquitectura **serverless** (sin servidores que mantener): todo corre en la infraestructura de Cloudflare y escala solo.

PARTE 3 — Tecnologías y servicios

Tecnologías (el "cómo está construido")

Tecnología	Qué es	Para qué la usamos	Ventaja
TypeScript	JavaScript con tipos	Todo el código (web y API)	Menos errores, más mantenible
React + Vite	Librería de interfaces + empaquetador	La web (catálogo, carrito, panel)	Rápido, moderno, gran comunidad
PWA	App web instalable	Que se pueda "instalar" y usar la cámara	Experiencia tipo app, sin tiendas

Tecnología	Qué es	Para qué la usamos	Ventaja
Hono	Framework web liviano	La API (rutas, lógica)	Hecho para Workers, muy veloz
Drizzle ORM	Capa de acceso a datos	Hablar con la base con tipos	Seguro y con migraciones
React Router	Navegación en la web	Las páginas (tienda, ficha, admin)	Navegación fluida sin recargar

Servicios (el "dónde vive y qué contrata")

Servicio	Qué es	Para qué lo usamos	Ventaja
Cloudflare Pages	Hosting de webs	Publicar la página	Gratis, rápido, global (CDN)
Cloudflare Workers	Funciones serverless	Correr la API	Sin servidor, escala solo
Cloudflare D1	Base de datos SQLite	Libros, usuarios, pedidos	Gratis hasta buen volumen
Cloudflare R2	Almacenamiento de archivos	Fotos de portadas/ejemplares	Sin costo de salida de datos
MercadoPago	Pasarela de pagos	Cobros con tarjeta/Yape	Líder en LatAm, confiable
NOWPayments	Pasarela de cripto	Cobros en BTC/ETH/USDT	Comisión baja, va a tu wallet
Lemon	Wallet/exchange cripto	Recibir y convertir tu cripto	Tu billetera de cobro
Telegram Bot	Mensajería	Avisos de venta al celular	Gratis e instantáneo
Resend	Envío de emails	Avisos de venta por correo	Gratis hasta 3.000/mes
Google Books API	Datos de libros	Autocompletar por ISBN	Gratis, gran cobertura

Seguridad y autenticación

- Login propio con contraseñas **cifradas (PBKDF2)** y **tokens de sesión**.
- Áreas privadas (admin) protegidas por rol.
- Pagos verificados por **webhooks con firma** (MercadoPago y NOWPayments) para que nadie pueda falsificar una confirmación de pago.

PARTE 4 — Procesos de pago (pasos que aplicamos)

Flujo general (igual para ambos métodos)

1. El cliente arma el **carrito** y completa **datos de envío** (nombre, teléfono, dirección).
2. Toca **Pagar** → la API crea un **pedido "pendiente"** y un cobro en la pasarela.

3. El cliente paga en la pantalla de la pasarela.
4. La pasarela avisa por **webhook** → la API confirma el pago → el pedido pasa a "**pagado**", baja el **stock** y se vacía el carrito.
5. Llegan los **avisos** (Telegram + correo + alerta en el panel).
6. El admin ve el pedido en "**Ventas**" con los datos de entrega y gestiona el envío (por atender → enviado → entregado).

MercadoPago (tarjetas, Yape, etc.)

- Integración **Checkout Pro**: se crea una "preferencia", el cliente paga y vuelve al sitio.
- **Credenciales de producción** cargadas como secreto (token real).
- Comisión: la descuenta MercadoPago de cada venta.

Cripto (NOWPayments → Lemon)

- Se crea una **factura**; el cliente paga BTC/ETH/USDT desde su wallet.
- El **precio en soles se convierte a USD** al cambio del día.
- El saldo entra a NOWPayments (modo **Custody**) y se **retira a Lemon** (USDT por Polygon).
- **Mínimo de S/ 50** para cripto: en montos chicos las comisiones fijas no convienen.
- Configuración clave: cobertura de pago 3% (tolera pequeñas diferencias), auto-conversión a USDT, optimización de comisiones de red.

PARTE 5 — Costos

Lo que es GRATIS (plan gratuito, para tu volumen)

- **Cloudflare Pages, Workers, D1, R2** → \$0 (las capas gratuitas cubren de sobra una tienda como esta).
- **Google Books API, Telegram, Resend (hasta 3.000 correos/mes)** → \$0.
- **NOWPayments**: sin costo fijo mensual (solo comisión por venta).
- **MercadoPago**: sin costo de alta ni mensualidad.

Costos variables (solo cuando vendés)

Concepto	Costo aprox.
Comisión MercadoPago (Perú)	~3,5% + IGV por venta
Comisión NOWPayments (cripto)	~0,5% + comisión de red
Comisión de wallet del cliente (cripto)	variable (la paga el comprador)

Costos fijos opcionales

Concepto	Costo aprox.
Dominio propio (.com / .pe)	~US\$ 10–15 al año (~S/ 40–55/año)
Cloudflare de pago (si crecés mucho)	desde US\$ 5/mes (no necesario hoy)

Costo operativo actual

~US\$ 0 al mes. Solo pagás **comisiones cuando vendés**, y opcionalmente el dominio (~US\$ 1/mes prorrateado). Es uno de los grandes beneficios de la arquitectura serverless.

Costo estimado de DESARROLLO (si se contratara)

Construir esta plataforma desde cero (catálogo, fotos, usuarios, carrito, **dos pasarelas de pago, **notificaciones, gestión de envíos, SEO, etc.**) es un **proyecto full-stack**** considerable.

Escenario	Estimado
Freelancer (LatAm)	US\$ 1.000 – 4.000 (≈ 60–120 h de trabajo)
Agencia / estudio	US\$ 5.000 – 15.000+
Lo que costó acá (con asistencia de IA)	prácticamente solo tu tiempo

El valor real de la plataforma es alto, pero al construirla con asistencia de IA el desembolso fue mínimo. El costo de **mantenerla** es casi nulo.

PARTE 6 — ¿Qué disciplina monta todo esto y qué hay que estudiar?

La disciplina

Esto lo arma un **Desarrollador Web Full-Stack** (programación de software), con toques de:

- **DevOps / Cloud** (para desplegar y configurar la infraestructura).
- **Marketing digital / SEO** (para el posicionamiento).

"Full-stack" = maneja tanto el **frente** (lo que ve el usuario) como el **fondo** (servidor, base de datos, integraciones).

Qué conocer / estudiar para integrar lo que usamos

Bases

- **HTML, CSS y JavaScript** → la web.
- **TypeScript** → JavaScript con tipos (lo que usamos).
- **Git** → control de versiones del código.

Frontend

- **React** y un empaquetador (**Vite**).
- Conceptos de **PWA**.

Backend / datos

- **APIs REST y HTTP** (rutas, métodos, JSON).
- **Bases de datos / SQL** y un **ORM** (Drizzle).
- **Serverless** y la plataforma **Cloudflare** (Workers, Pages, D1, R2).

Integraciones y seguridad

- **Pasarelas de pago y webhooks** (MercadoPago, NOWPayments).
- Nociones de **seguridad** (hashing de contraseñas, tokens, firmas).
- **APIs de terceros** (Google Books, Telegram, Resend).

Difusión

- **SEO** y herramientas como Google Search Console.

Ruta sugerida de estudio

1. HTML/CSS/JS → 2. TypeScript → 3. React → 4. APIs/HTTP → 5. Bases de datos/SQL →
6. Serverless + Cloudflare → 7. Integración de pagos y webhooks → 8. SEO.

No hace falta dominar todo para operar la tienda: con esta documentación podés **gestionarla**.

Lo de arriba es lo que necesitaría alguien para **construir o modificar** la plataforma.

Resumen de una línea

Una **tienda de libros serverless** en Cloudflare (web React + API Hono + base D1 + fotos R2), con **pagos por MercadoPago y cripto**, **avisos** por Telegram/correo, **SEO** completo, y un costo operativo de **casi US\$ 0/mes**.